



Exercício: Gerenciador de Veículos Polimórficos

Objetivo

O objetivo deste exercício é demonstrar o uso de polimorfismo em uma hierarquia de classes relacionadas a veículos.

Instruções

Passos para fazer o exercício:

1. Definir a classe abstrata **Veiculo**:

- A classe **Veiculo** é abstrata, o que significa que não pode ser instanciada diretamente. Ela serve como um modelo para outras classes derivadas.
- Ela possui dois atributos privados: **placa** (String) e **ano** (int).
- O construtor da classe **Veiculo** recebe esses dois parâmetros e inicializa os atributos.
- A classe **Veiculo** também possui um método chamado **exibirInformacoes()**, que imprime a placa e o ano do veículo.

2. Criar as classes derivadas **Onibus** e **Caminhao**:

- Ambas as classes herdam da classe **Veiculo**.
- A classe **Onibus** possui um atributo adicional chamado **assentos** (int).
- A classe **Caminhao** possui um atributo adicional chamado **eixos** (int).
- Os construtores das classes derivadas chamam o construtor da classe base (**Veiculo**) usando **super(placa, ano)**.
- Cada classe derivada sobrescreve o método **exibirInformacoes()** para incluir informações específicas (assentos para ônibus e eixos para caminhões).

3. Classe **GerenciadorVeiculos**:

- Essa classe contém o método **main**, que é o ponto de entrada do programa.
- No método **main**, você cria instâncias de **Onibus** e **Caminhao** com valores específicos para placa, ano, assentos e eixos.
- Em seguida, você chama o método **exibirInformacoes()** para mostrar os detalhes de cada veículo.

Lembre-se de que, como a classe **Veiculo** é abstrata, você não pode criar um objeto diretamente dela. Use as classes derivadas (**Onibus** e **Caminhao**) para instanciar veículos específicos e exibir suas informações.

Código do exercício:

```
abstract class Veiculo {
    private String placa;
    private int ano;

    public Veiculo(String placa, int ano) {
        this.placa = placa;
        this.ano = ano;
    }

    // Novo método para exibir informações
    public void exibirInformacoes() {
        System.out.println("Placa: " + placa);
        System.out.println("Ano: " + ano);
    }
}

class Onibus extends Veiculo {
    private int assentos;

    public Onibus(String placa, int ano, int assentos) {
        super(placa, ano);
        this.assentos = assentos;
    }

    @Override
    public void exibirInformacoes() {
        super.exibirInformacoes();
        System.out.println("Assentos: " + assentos);
    }
}

class Caminhao extends Veiculo {
    private int eixos;

    public Caminhao(String placa, int ano, int eixos) {
        super(placa, ano);
        this.eixos = eixos;
    }

    @Override
    public void exibirInformacoes() {
        super.exibirInformacoes();
        System.out.println("Eixos: " + eixos);
    }
}
```

```
public class GerenciadorVeiculos {  
    public static void main(String[] args) {  
        Veiculo veiculo1 = new Onibus("ABC123", 2020, 40);  
        Veiculo veiculo2 = new Caminhao("XYZ789", 2018, 6);  
  
        System.out.println("Detalhes do veículo 1 (ônibus):");  
        veiculo1.exibirInformacoes();  
        System.out.println();  
  
        System.out.println("Detalhes do veículo 2 (caminhão):");  
        veiculo2.exibirInformacoes();  
    }  
}
```

Explicação do Código

Neste exercício, você pode observar como o polimorfismo permite que os métodos `exibirInformacoes()` se comportem de maneira diferente nas subclasses `Onibus` e `Caminhao`, mesmo sendo chamados através de referências do tipo `Veiculo`. Isso demonstra a flexibilidade e a capacidade de reutilização de código proporcionada pelo polimorfismo em Java.



Exercício: Calculadora Polimórfica

Objetivo:

O objetivo deste exercício é criar uma calculadora polimórfica em Java que pode realizar operações de adição, subtração, multiplicação e divisão.

Instruções

1. Crie uma classe abstrata chamada `Operacao` com um método abstrato `calcular`.
2. Implemente a classe `Operacao` em quatro classes concretas: `Soma`, `Subtracao`, `Multiplicacao` e `Divisao`.
3. Crie uma classe chamada `Calculadora` com o método `main` para testar as implementações das operações matemáticas.
4. Compile e execute o programa para verificar os resultados.

Passos para Fazer o Exercício

1. Criação da Classe Abstrata `Operacao`:

- Defina a classe abstrata `Operacao` com o método abstrato `calcular` que recebe dois parâmetros do tipo `double`.

2. Implementação das Classes Concretas:

- Crie a classe `Soma` que estende `Operacao` e implemente o método `calcular` para realizar a soma.
- Crie a classe `Subtracao` que estende `Operacao` e implemente o método `calcular` para realizar a subtração.
- Crie a classe `Multiplicacao` que estende `Operacao` e implemente o método `calcular` para realizar a multiplicação.
- Crie a classe `Divisao` que estende `Operacao` e implemente o método `calcular` para realizar a divisão, incluindo uma verificação para divisão por zero.

3. Criação da Classe `Calculadora`:

- Defina a classe `Calculadora`.
- No método `main`, crie instâncias de `Soma`, `Subtracao`, `Multiplicacao` e `Divisao`.
- Chame o método `calcular` para cada operação com valores de exemplo.
- Imprima os resultados no console.

4. Compilação e Execução:

- Compile o código utilizando um compilador Java.

- Execute o programa e verifique os resultados impressos no console.

Código do Exercício

```
abstract class Operacao {
    public abstract double calcular(double a, double b);
}

class Soma extends Operacao {
    @Override
    public double calcular(double a, double b) {
        return a + b;
    }
}

class Subtracao extends Operacao {
    @Override
    public double calcular(double a, double b) {
        return a - b;
    }
}

class Multiplicacao extends Operacao {
    @Override
    public double calcular(double a, double b) {
        return a * b;
    }
}

class Divisao extends Operacao {
    @Override
    public double calcular(double a, double b) {
        if (b != 0) {
            return a / b;
        } else {
            System.out.println("Erro: Divisão por zero não é permitida.");
            return Double.NaN; // Retorna NaN (Not-a-Number) para representar erro
        }
    }
}

public class Calculadora {
    public static void main(String[] args) {
        Operacao soma = new Soma();
        Operacao subtracao = new Subtracao();
        Operacao multiplicacao = new Multiplicacao();
        Operacao divisao = new Divisao();

        double num1 = 10;
        double num2 = 5;

        System.out.println("Soma: " + soma.calcular(num1, num2));
    }
}
```

```
        System.out.println("Subtração: " + subtracao.calcular(num1,
num2));
        System.out.println("Multiplicação: " +
multiplicacao.calcular(num1, num2));
        System.out.println("Divisão: " + divisao.calcular(num1,
num2));
    }
}
```

Explicação do Código

- **Classe Abstrata `Operacao`:**

- Define o método abstrato `calcular` que deve ser implementado pelas subclasses para realizar operações matemáticas.

- **Classes `Soma`, `Subtracao`, `Multiplicacao` e `Divisao`:**

- Cada uma dessas classes estende `Operacao` e implementa o método `calcular` para realizar a operação matemática correspondente.
- A classe `Divisao` inclui uma verificação para evitar divisão por zero, retornando `NaN` e exibindo uma mensagem de erro se `b` for zero.

- **Classe `Calculadora`:**

- No método `main`, são criadas instâncias de `Soma`, `Subtracao`, `Multiplicacao` e `Divisao`.
- O método `calcular` é chamado para cada operação com os valores `10` e `5`.
- Os resultados das operações são impressos no console.

Este exercício ajuda a entender como criar e utilizar classes abstratas e métodos abstratos em Java, além de praticar a implementação de operações matemáticas básicas.